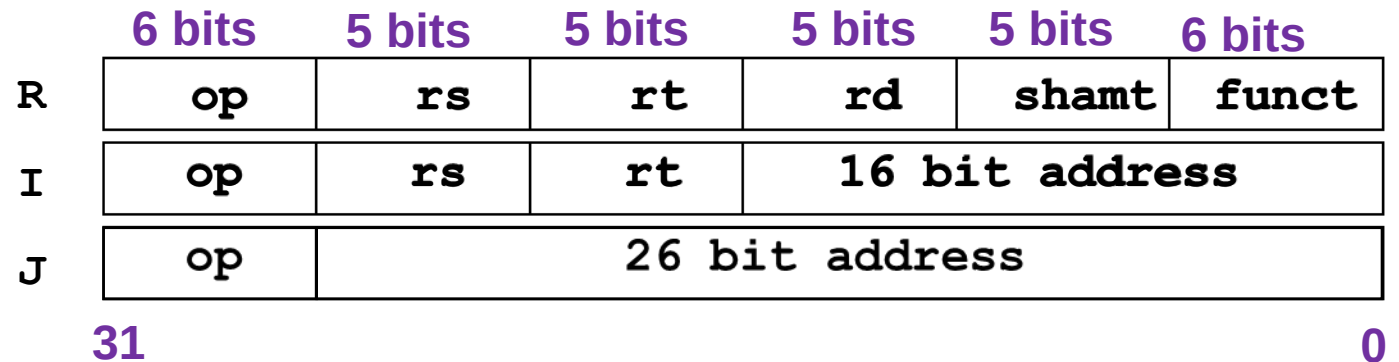


MIPS MACHINE LANGUAGE

- Computers only understand 1's and 0's
- Machine language: binary representation of assembly instructions
- 32-bit instructions
 - Again **DP1: Simplicity favors regularity**
 - 32-bit data, 32-bit instructions, and possibly also 32-bit addresses

- Three instruction formats:

- **R-Type**: register operands
- **I-Type**: immediate operand
- **J-Type**: Jump type



(R) Register only; most of the arithmetic, jump register, syscall instructions

(I) Arithmetic, logical instructions that use immediate, branch instructions

(J) Jump instructions that use memory address; mostly the jump instructions.

THREE MIPS INSTRUCTION FORMATS

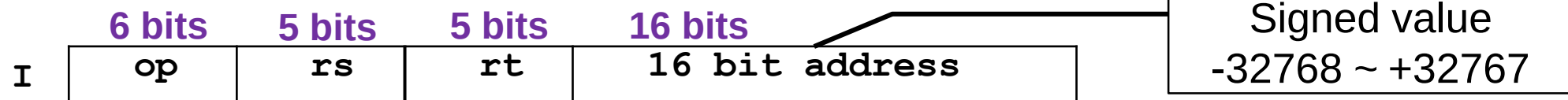
- ❑ simple instructions all 32 bits wide
- ❑ only three instruction formats
- ❑ very structured, no unnecessary baggage

Simplicity favors regularity

Consistent instruction format: same no. of operands (2 source, 1 destination) → easier to encode and handle in hardware.



- ❑ **R-type:** ALU instructions (add, sub,...), shift (srl, sll), jump register (jr ...),
- ❑ May use 3 registers (most instructions) or 2 registers (sll, srl) or only 1 register (jr, jalr)



- ❑ **I-type:** immediate (addi ...), load (lw ...), store (sw ...), conditional branches (bne ...)
- ❑ rt is the destination and rs is the source register.



- ❑ **J-type:** jump (j), jump and link (jal) → uses absolute value 26-bit.
- ❑ Address space is 2^{28} . (Absolute address in a 256MB block)

op=0 → (R) type Instruction
op>3; → (I) type Instruction
op = 2 or 3 → (J) type Instruction

MIPS INSTRUCTIONS: R-TYPE

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

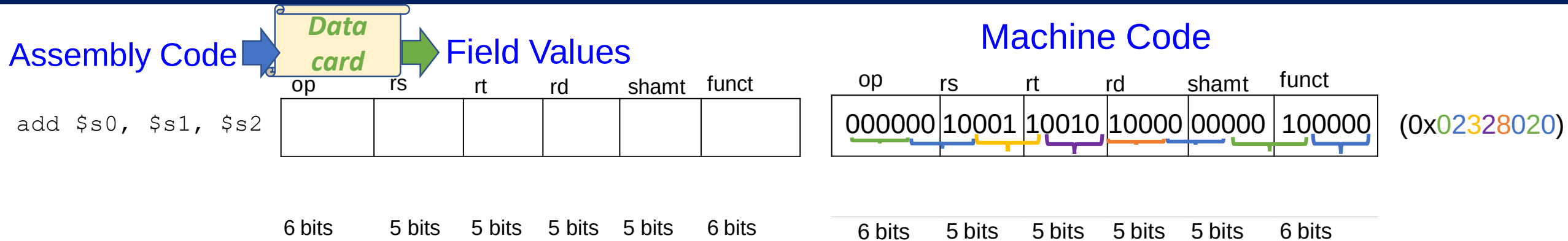
Register-type, 3 register operands:

- **rs, rt**: source registers
- **rd**: destination register

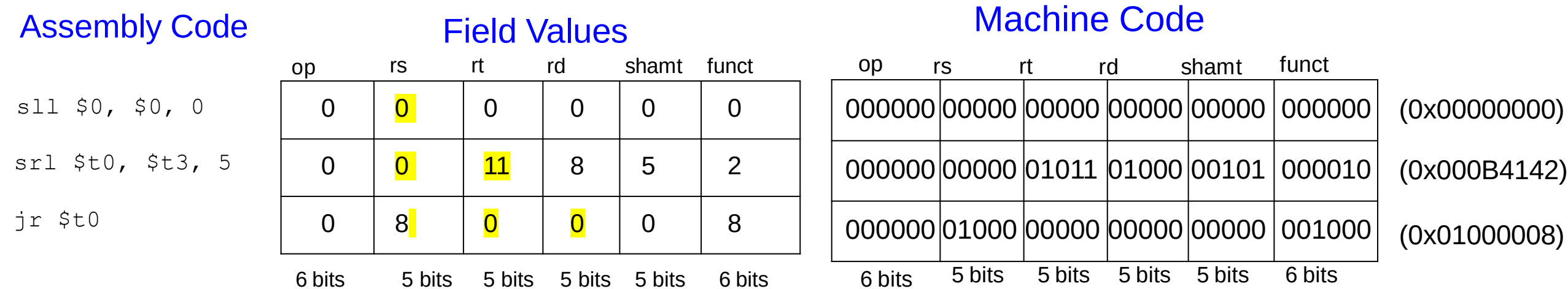
Other fields:

- **op**: the operation code or opcode (0 for R-type instructions)
- **funct**: the function → the opcode and function together tell the computer what operation to perform.
- **shamt**: the shift amount for shift instructions, otherwise it's 0

R-TYPE EXAMPLES (ASSEMBLY TO MACHINE CODE)



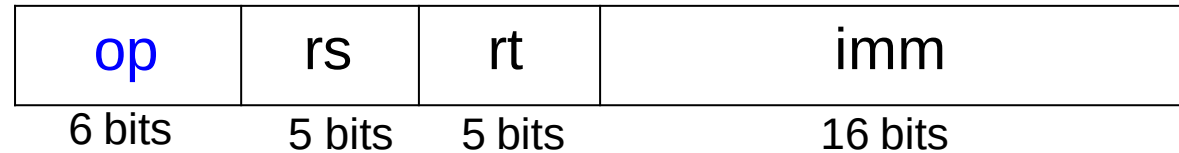
Note: the change in order of registers: AC: `add rd, rs, rt` → MC: `rs, rt, rd`



Note: the register `rs` is unused in the assembly code: `srl rd, rs=0, rt, shamt`;
 and registers `rt` and `rd` are unused in the assembly code: `jr (rs=$t0)`

MIPS INSTRUCTIONS: I-TYPE

I-Type



Immediate-type, has 3 operands:

- **rs**, register operands (rs = source, rt = destination)
- **rt**: 16-bit two's complement (immediate value)
- **imm**

Other fields:

- **op**: the opcode

Simplicity favors regularity: all instructions have opcode

Instruction type is completely determined by the opcode

I-TYPE EXAMPLES

Assembly Code

```
addi $s0, $s1, 5
addi $t0, $s3, -12
lw    $t2, 32($0)
sw    $s1, 4($t1)
```

Field Values

op	rs	rt	imm
8	17	16	5
8	19	8	-12
35	0	10	32
43	9	17	4
6 bits	5 bits	5 bits	16 bits

Machine Code

op	rs	rt	imm	
001000	10001	10000	0000 0000 0000 0101	(0x22300005)
001000	10011	01000	1111 1111 1111 0100	(0x2268FFF4)
100011	00000	01010	0000 0000 0010 0000	(0x8C0A0020)
101011	01001	10001	0000 0000 0000 0100	(0xAD310004)
6 bits	5 bits	5 bits	16 bits	

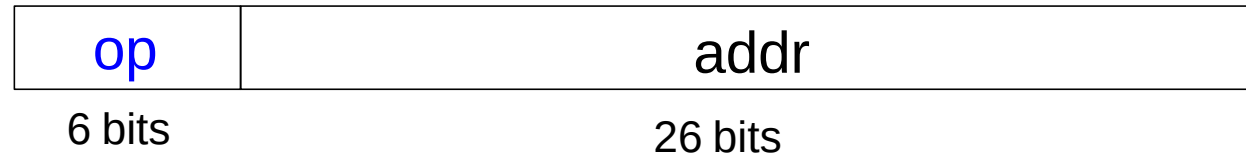
Note the differing order of registers in the assembly and machine codes:

```
addi $s0, $s1, 5
addi $t0, $s3, -12
lw    $t2, 32($0)
sw    $s1, 4($t1)
```

```
addi    rt,    rs, imm
lw      rt,    imm(rs)
sw      rt,    imm(rs)
```

MACHINE LANGUAGE: J-TYPE

J-Type



Jump-type

26-bit address operand (address)

Used for jump instructions:

1. j → op = 2
2. jal → op = 3

REVIEW: INSTRUCTION FORMATS

R-Type

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

I-Type

op	rs	rt	imm
6 bits	5 bits	5 bits	16 bits

J-Type

op	addr
6 bits	26 bits

op = 0

op > 3

op = 2 or 3

DESIGN PRINCIPLE 4

Good design demands good compromises

- **Multiple instruction formats allow flexibility**

- **add, sub:** use 3 register operands
- **lw, sw:** use 2 register operands and a constant

- **Number of instruction formats kept small**

- to adhere to design principles 1 and 3 (simplicity favors regularity and smaller is faster)

DISASSEMBLING - PARSING THE MIPS MACHINE INSTRUCTIONS

Machine Code

Field Values

Assembly Code

(0x2237FFF1)

op	rs	rt	imm			
001000	10001	10111	1111	1111	1111	0001
2	2	3	7	F	F	F 1

(0x02F34022)

op	rs	rt	rd	shamt	funct
000000	10111	10011	01000	00000	100010
0	2	F	3	4	0 2 2

op	rs	rt	imm			
8	17	23	-15			

addi \$s7, \$s1, -15

op	rs	rt	rd	shamt	funct
0	23	19	8	0	34

sub \$t0, \$s7, \$s3

- **Start with opcode**
 - Opcode tells how to parse the remaining bits
- **If opcode is all 0's**
 - R-type instruction
 - Function bits tell what instruction it is
- **Otherwise**
 - opcode tells what instruction it is

MIPS INSTRUCTION SUMMARY

#	Instruction	Opcode/Func	Usage
1	add (R)	0/32	add \$rd, \$rs, \$rt
2	addu (R)	0/33	addu \$rd, \$rs, \$rt
3	sub (R)	0/34	sub \$rd, \$rs, \$rt
4	subu (R)	0/35	subu \$rd, \$rs, \$rt
5	and (R)	0/36	and \$rd, \$rs, \$rt
6	or (R)	0/37	or \$rd, \$rs, \$rt
7	xor (R)	0/38	xor \$rd, \$rs, \$rt
8	nor (R)	0/39	nor \$rd, \$rs, \$rt
9	sll (R)	0/0	sll \$rd, \$rt, #v (no \$rs)
10	srl (R)	0/2	srl \$rd, \$rt, #v
11	sra (R)	0/3	sra \$rd, \$rt, #v

#	Instruction	Opcode/Func	Usage
12	slt (R)	0/42	slt \$rd, \$rs, \$rt
13	sltu (R)	0/43	sltu \$rd, \$rs, \$rt
14	mult (R)	0/24	mult \$rs, \$rt (no \$rd)
15	div (R)	0/26	div \$rs, \$rt (no \$rd)
16	mfhi	0/16	mfhi \$rd (no \$rs \$rt)
17	mflo	0/18	mflo \$rd (no \$rs \$rt)
18	jr (R)	0/8	jr \$rs (no \$rt \$rd)
19	jalr (R)	0/9	jalr \$rd* \$rs, (opt*)

\$rd is always the destination register.
 \$rs and \$rt are always the source registers.
 \$rs is unused in Shift/mfhi/mflo instructions
 \$rt and \$rd are unused in jr instruction
 \$rd* input is optional in jalr (defaults to \$ra)

Rule of Thumb: Unused registers are always set to 0.

MIPS INSTRUCTION SUMMARY

#	Instruction	Opcode/Func	Usage
21	addi (I)	8/0	addi \$rt, \$rs, #v
22	addiu (I)	9/0	addiu \$rt, \$rs, #v
23	andi (I)	12/0	andi \$rt, \$rs, #v
24	ori (I)	13/0	ori \$rt, \$rs, #v
25	xori (I)	14/0	xori \$rt, \$rs, #v
26	lui (I)	15/0	lui \$rt, 0, #v
27	slti (I)	10/38	slti \$rt, \$rs, #v
28	sltiu (I)	11/39	sltiu \$rt, \$rs, #v
29	beq (I)	4/0	beq \$rs, \$rt, #v
30	bne (I)	5/0	bne \$rs, \$rt, #v
31	ll (I)	48/0	ll \$rt, \$rs, #v

#	Instruction	Opcode/Func	Usage
32	lw (I)	35/0	lw \$rt, \$rs, #v
33	lh (I)	33/0	lh \$rt, \$rs, #v
34	lhu (I)	37/0	lhu \$rt, \$rs, #v
35	lb (I)	32/0	lb \$rt, \$rs, #v
36	lbu (I)	36/0	lbu \$rt, \$rs, #v
37	sw (I)	43/0	sw \$rt, \$rs, #v
38	sh (I)	41/0	sh \$rt, \$rs, #v
39	sb (I)	40/0	sb \$rt, \$rs, #v
40	sc (I)	48/0	sc \$rt, \$rs, #v
41	j (J)	2/0	j #v
42	jal (J)	3/0	jal #v

\$rs is always the source registers.

\$rt is always the destination register except in branch instructions where it acts as 2nd source register.

\$rs is unused in lui instruction